Visual Recognition Mini Project - Part 1

Multi-Object Apparel Detection and Instance Segmentation

Group Members:

Saheem Showkat Reshi (IMT2023051)
Ayush Mishra (IMT2023129)
Harsh Sinha (IMT2023571)
Nipun Verma (IMT2023591)

Resources:

GitHub Repository

Hugging Face Model

Google Drive (All Models)

Course: Visual Recognition

March 31, 2026

Contents

1 Introduction & Problem Statement **4**

2 Implementation Notes and Additional Details **4**

3 Dataset Description and Exploratory Data Analysis **5**
- 3.1 Dataset Statistics . . . 5
- 3.2 Class Distribution . . . 5
- 3.3 Visual Analysis . . . 6
- 3.4 Key Observations . . . 7
- 3.5 Dataset Curation . . . 7

4 Data Preprocessing **8**
- 4.1 Dataset Insights from EDA . . . 8
- 4.2 Class Imbalance Analysis . . . 8
- 4.3 Handling Imbalance and Data Augmentation . . . 9
- 4.4 Image Preprocessing Pipeline . . . 9
- 4.5 Data Splits . . . 9

5 Experimental Setup **10**
- 5.1 Hardware and Training Environment . . . 10
- 5.2 Task 3.1: Classification Architectures . . . 10
  - 5.2.1 ResNet-50 . . . 10
  - 5.2.2 EfficientNet-B0 . . . 10
  - 5.2.3 MobileNetV3 . . . 10
- 5.3 Task 3.2: Detection and Segmentation Architectures . . . 10
  - 5.3.1 YOLO . . . 11
  - 5.3.2 Mask R-CNN . . . 11
  - 5.3.3 U-Net . . . 11

6 Results **12**
- 6.1 Classification Results . . . 12
  - 6.1.1 Overall Performance . . . 12
  - 6.1.2 Per-Class Performance . . . 13
  - 6.1.3 ROC and AUC Curves . . . 13
- 6.2 Detection Results . . . 15
  - 6.2.1 Overall Performance . . . 15
  - 6.2.2 Per-Class Detection Performance . . . 15
- 6.3 Segmentation Results . . . 16
  - 6.3.1 Overall Performance . . . 16
  - 6.3.2 Per-Class Segmentation Performance . . . 17
  - 6.3.3 ROC and AUC Curves for Detection Models . . . 17
- 6.4 Qualitative Results . . . 18

7 Comparison and Analysis **19**
7.1 Classification Analysis 19
7.2 Detection Analysis 20
7.3 Segmentation Analysis 20

8 Discussion and Insights **20**
8.1 Failure Cases 21
8.2 Key Learnings 22

9 Conclusion **22**

1 Introduction & Problem Statement

With the rapid growth of fashion e-commerce and digital retail platforms, there is an increasing demand for automated visual recognition systems capable of understanding clothing items in images. Such systems are essential for applications like product catalog generation, visual search, recommendation systems, and inventory management. However, analyzing clothing in real-world images is inherently challenging due to factors such as high intra-class variation, occlusions, varying viewpoints, and the deformable nature of garments.

In this project, we aim to design a robust visual recognition pipeline using a subset of the DeepFashion2 dataset. The system is required to handle images containing multiple clothing items belonging to different categories, making the problem significantly more complex than traditional single-label classification tasks.

To address this, the project is formulated into two primary computer vision tasks:

- **Multi-Label Image Classification:** Given an input image, the model predicts the presence or absence of each clothing category. Since multiple clothing items can coexist in a single image, this is treated as a multi-label classification problem rather than a mutually exclusive multi-class setting.

- **Object Detection and Instance Segmentation:** The model must localize each clothing item using bounding boxes and generate precise pixel-level masks for every detected instance. This requires simultaneous learning of localization, classification, and segmentation.

The problem is further constrained by practical considerations such as limited computational resources, class imbalance in the dataset, and the need to balance model complexity with efficiency. These challenges necessitate careful model selection, data preprocessing, and training strategies to achieve optimal performance.

Overall, this project aims to build an end-to-end system capable of understanding complex, multi-object fashion images while evaluating different architectures under both training-from-scratch and transfer learning paradigms.

2 Implementation Notes and Additional Details

- **Initial transfer learning attempt (frozen backbone):** We initially experimented with basic transfer learning where the entire backbone was frozen and only the final classification layer was trained. However, this approach resulted in poor performance, as the models were unable to adapt to domain-specific features of clothing images. These experiments are available under the name *basicTL* in both the Drive and GitHub folders.

- **Improved transfer learning with fine-tuning:** To address the above issue, we adopted a fine-tuning strategy. Initially, only the last few layers were unfrozen to allow gradual adaptation to the dataset. Subsequently, the entire network was unfrozen and trained with a lower learning rate. This approach significantly improved performance, as it allowed the model to adjust both high-level and mid-level representations to the fashion domain. Additionally, a multi-label setup using BCEWithLogitsLoss was used to handle multiple clothing categories per image. All transfer learning results reported in the classification section correspond to this fine-tuning setup (TL + FT), and not the basic frozen-backbone approach.

- **Metrics not applicable to U-Net:**

Certain evaluation metrics were not applied to U-Net due to its formulation as a semantic segmentation model.

- **ROC curves and AUC:** ROC curves were not generated for U-Net since it operates at the pixel level and does not produce confidence scores for object-level predictions. Metrics such as mIoU and Dice coefficient are more appropriate for evaluating segmentation quality.
- **Macro and Micro F1-score (classification):** These metrics are designed for multi-label image-level classification tasks. Since U-Net performs pixel-wise segmentation rather than predicting image-level class labels, these metrics are not applicable.
- **COCO-style mAP@[0.5:0.95]:** This metric requires confidence scores and ranked predictions for detected objects. U-Net does not directly output instance-level detections or confidence scores; instead, object instances are derived via post-processing (connected component analysis). Therefore, true COCO mAP cannot be reliably computed.
- **ROC/AUC for detection:** Similar to mAP, detection-based ROC/AUC analysis relies on confidence scores for bounding box predictions. Since U-Net does not provide such scores, ROC-based detection evaluation is not meaningful.

Instead, U-Net is evaluated using segmentation-specific metrics such as mean Intersection over Union (mIoU) and Dice coefficient, along with approximate detection metrics (Precision, Recall, and F1-score) derived from post-processed instance predictions.

3 Dataset Description and Exploratory Data Analysis

We utilize the DeepFashion2 dataset, which consists of person-centric images annotated with rich semantic information including category labels, bounding boxes, and segmentation masks. To better understand the dataset characteristics and distribution, we perform an exploratory data analysis (EDA) on the training split.

3.1 Dataset Statistics

The dataset exhibits the following overall statistics:

- **Total Images:** 191,961
- **Total Objects:** 312,186
- **Average Objects per Image:** 1.63

These statistics indicate that most images contain a small number of objects, making the dataset moderately sparse in terms of object density.

3.2 Class Distribution

The distribution of the most frequent categories in the dataset is shown in Table 1. It is evident that the dataset is imbalanced, with certain categories appearing significantly more often than others.

Category	Count
Short Sleeve Top	71,645
Trousers	55,387
Shorts	36,616
Long Sleeve Top	36,064
Skirt	30,835
Vest Dress	17,949
Short Sleeve Dress	17,211
Vest	16,095
Long Sleeve Outwear	13,457
Long Sleeve Dress	7,907

Table 1: Top 10 class distribution in the dataset

3.3 Visual Analysis

Figure 1 illustrates the class distribution, highlighting the imbalance across categories.

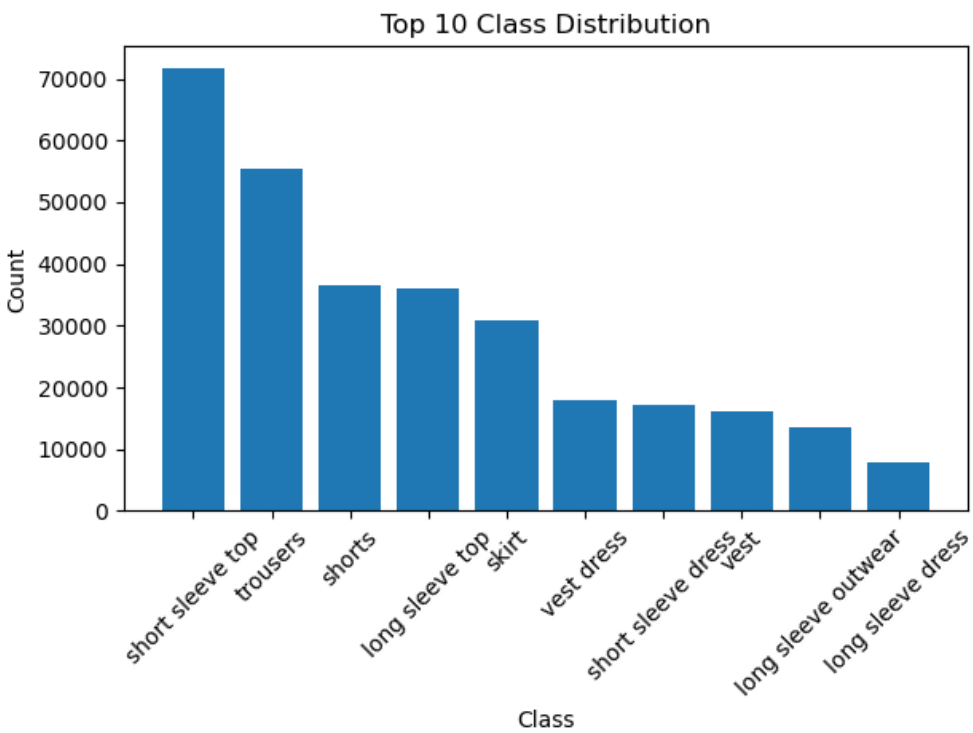


Figure 1: Top 10 class distribution

Figure 2 shows the distribution of the number of objects per image. Most images contain one or two objects, with very few images containing more than three objects.

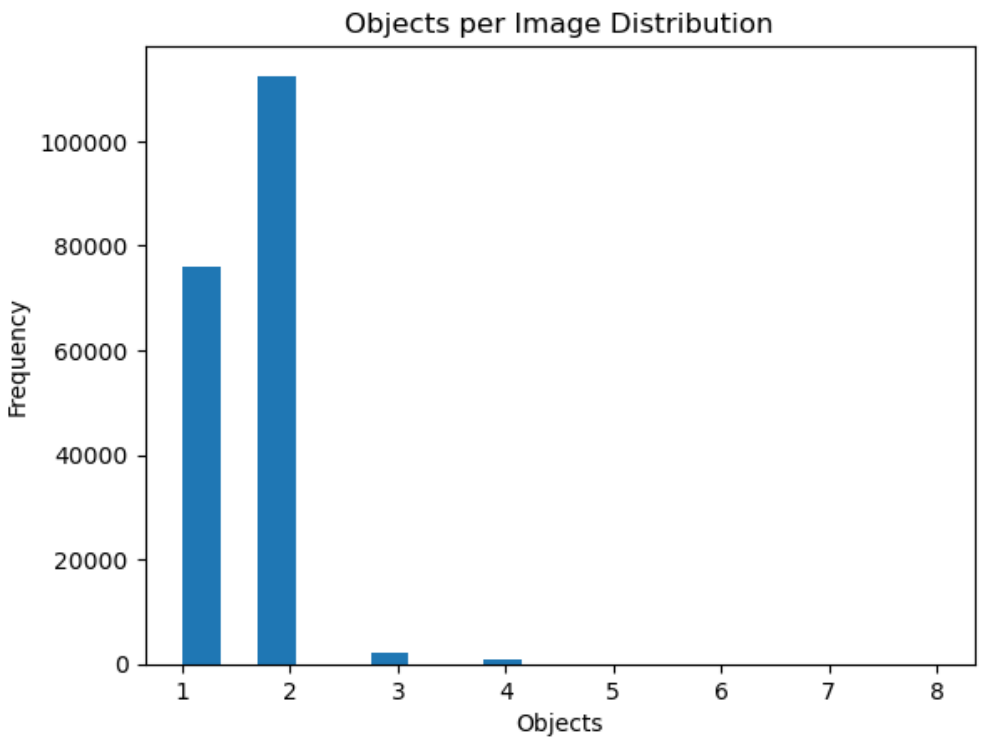


Figure 2: Distribution of objects per image

3.4 Key Observations

- The dataset is **class-imbalanced**, with top categories dominating the distribution.
- Most images contain **1–2 objects**, indicating relatively simple scenes.
- Rare classes may be harder to learn due to limited representation.

3.5 Dataset Curation

The dataset was curated by selecting the top 5 most frequent clothing categories, as required. All annotations were filtered to retain only these categories. We selected the top 5 categories based on frequency:

- Category 1: short sleeve top (71645)
- Category 2: trousers (55387)
- Category 3: shorts (36616)
- Category 4: long sleeve top (36064)
- Category 5: skirt (30835)

Different dataset strategies were adopted for classification and detection/segmentation tasks to balance performance and computational constraints:

- **Classification:** All three classification models were trained using the full dataset to maximize representation and improve generalization. The dataset consisted of 144,174 training images and 23,741 validation images. Final evaluation metrics were computed on a held-out test set of 23,741 images.
- **Detection & Segmentation:** Due to higher computational requirements, a subset of the dataset was used. All samples were drawn from the original training split. A random split was created and saved in `data_split.json`, which was consistently used for training, validation, and testing across all three segmentation models. The subset contained a total of 43,252 images, split as follows:
  - Training: 34,601 images
  - Validation: 4,325 images
  - Test: 4,326 images

4 Data Preprocessing

4.1 Dataset Insights from EDA

From the exploratory data analysis, the dataset exhibits the following characteristics:

- **Total Images:** 191,961
- **Total Objects:** 312,186
- **Average Objects per Image:** 1.63

The dataset primarily contains images with one or two objects, indicating relatively simple scenes. However, the class distribution is highly imbalanced, as shown in Figure 1.

4.2 Class Imbalance Analysis

The distribution of the top categories is skewed, with certain classes dominating the dataset:

- Short sleeve top: 71,645
- Trousers: 55,387
- Shorts: 36,616
- Long sleeve top: 36,064
- Skirt: 30,835

This imbalance can negatively impact model performance, especially for underrepresented classes.

4.3 Handling Imbalance and Data Augmentation

To improve generalization and mitigate class imbalance, we applied several data augmentation techniques during training:

- Random horizontal flipping
- Random rotations (up to 10–15 degrees)
- Color jittering (brightness and contrast variations)
- Random cropping

These augmentations help the model learn robust features by exposing it to variations in orientation, lighting, and scale.

4.4 Image Preprocessing Pipeline

After experimentation with multiple configurations, the final preprocessing pipeline used for training and validation is defined as follows:

Training Transformations:

- Resize images to 256×256
- Random crop to 224×224
- Random horizontal flip
- Convert to tensor
- Normalize using ImageNet statistics

Validation Transformations:

- Resize images to 256×256
- Center crop to 224×224
- Convert to tensor
- Normalize using ImageNet statistics

Normalization is performed using the standard ImageNet mean and standard deviation:

$$\mu = [0.485, 0.456, 0.406], \quad \sigma = [0.229, 0.224, 0.225]$$

4.5 Data Splits

The dataset is divided into training and validation sets as follows:

- **Training Set:** 144,174 images
- **Validation Set:** 23,741 images

This split ensures sufficient data for learning while maintaining a reliable validation set for evaluation.

5 Experimental Setup

This section details the experimental configurations, architectural adaptations, and training strategies used across all models. To evaluate the impact of initialization, each model was trained under two paradigms: training from scratch and transfer learning.

5.1 Hardware and Training Environment

All experiments were conducted using Kaggle GPU environments. Training was performed using T4x2 GPU.

5.2 Task 3.1: Classification Architectures

For the multi-label classification task, all models were adapted to output 5 logits corresponding to the selected clothing categories. A sigmoid activation was applied, and optimization was performed using Binary Cross-Entropy with Logits Loss.

5.2.1 ResNet-50

ResNet-50 was used as a deep baseline model.
From Scratch: Initialized with random weights and trained end-to-end using full backpropagation, requiring careful optimization due to slower convergence in deep architectures.
Transfer Learning: Initialized with ImageNet pre-trained weights, with backbone layers frozen initially and only the classification head trained. Subsequently, full fine-tuning was performed across all layers to improve domain adaptation, resulting in faster convergence and improved performance.

5.2.2 EfficientNet-B0

EfficientNet-B0 was used to evaluate parameter-efficient scaling.
From Scratch: Trained with random initialization using full backpropagation across all layers, along with standard augmentations and batch-based optimization.
Transfer Learning: Initialized with pre-trained weights and trained by freezing feature extraction layers while fine-tuning only the classification head, reducing computational cost but limiting domain adaptation.

5.2.3 MobileNetV3

MobileNetV3 was used as a lightweight architecture.
From Scratch: The model was initialized without pre-trained weights and trained fully end-to-end, allowing complete domain adaptation at the cost of higher computational requirements.
Transfer Learning: ImageNet pre-trained weights were utilized to initialize the network, with backbone layers frozen to retain learned feature representations. Training was limited to the final classification layer, enabling faster convergence but reducing domain-specific adaptability.

5.3 Task 3.2: Detection and Segmentation Architectures

For this task, models were required to jointly perform detection and segmentation. Three architectures were evaluated: YOLO, Mask R-CNN, and U-Net.

5.3.1 YOLO

YOLOv8n-seg was used as a single-stage architecture for joint detection and segmentation, operating on polygon-based annotations converted to YOLO segmentation format.

Training Configuration:

- Batch size: 8 (Scratch), 4 (Transfer)
- Epochs: 8 (Scratch), 5 (Transfer)
- Image size: 416×416

From Scratch: Initialized using the YOLOv8n-seg configuration (`pretrained=False`) and trained end-to-end on the custom dataset with polygon annotations converted to YOLO format.

Transfer Learning: Initialized with COCO pre-trained YOLOv8n-seg weights, with the first 10 layers frozen and remaining layers fine-tuned. Memory-intensive augmentations (mosaic, mixup, copy-paste) were disabled and image size reduced to 416 to ensure stable training on limited GPU resources. This YOLO Model is uploaded on the google drive link mentioned because the ipynb file was too big for github.

5.3.2 Mask R-CNN

Mask R-CNN was implemented as a two-stage detection and segmentation model with a ResNet-50 FPN backbone for multi-scale feature extraction.

Architecture: The model was adapted to the 5 selected clothing classes by replacing both the classification and mask prediction heads to match the required number of classes.

From Scratch: Initialized with random weights (`weights=None`), the model was trained end-to-end using full backpropagation. To address GPU memory constraints, images and masks were resized to 512×512, and mixed precision training with gradient accumulation was employed for stable optimization.

Transfer Learning: Initialized with COCO pre-trained weights, with the backbone frozen and only ROI heads trained. This reduced computational cost and enabled faster convergence while retaining general visual features.

5.3.3 U-Net

U-Net was used as the primary architecture for semantic segmentation.

Architecture: An encoder–decoder structure with skip connections was employed to capture both high-level semantic features and fine spatial details. A ResNet-34 backbone was used as the encoder.

Training: Two settings were explored:

- *From Scratch:* Entire network initialized randomly and trained end-to-end.
- *Transfer Learning:* Encoder initialized with ImageNet pre-trained weights, while the decoder was trained from scratch.

Preprocessing: Images were resized to 256×256, normalized using ImageNet statistics, and augmented with flipping, rotation, and brightness/contrast changes.

Loss and Optimization: A combination of weighted Cross-Entropy Loss and Dice Loss (excluding background) was used. Training was performed using the Adam optimizer with early stopping based on validation IoU.

Evaluation: Performance was measured using mIoU and Dice coefficient over foreground classes.

6 Results

6.1 Classification Results

6.1.1 Overall Performance

Model	Training	Precision	Recall	Macro F1	Micro F1
ResNet-50	Scratch	0.7897	0.7317	0.7596	0.7389
ResNet-50	Transfer	0.8679	0.8263	0.8465	0.8590
EfficientNet-B0	Scratch	0.6968	0.8536	0.7617	0.7813
EfficientNet-B0	Transfer	0.8763	0.8516	0.8515	0.8638
MobileNetV3	Scratch	0.6967	0.8170	0.7487	0.7669
MobileNetV3	Transfer	0.8635	0.8156	0.8215	0.8388

Table 2: Overall classification performance

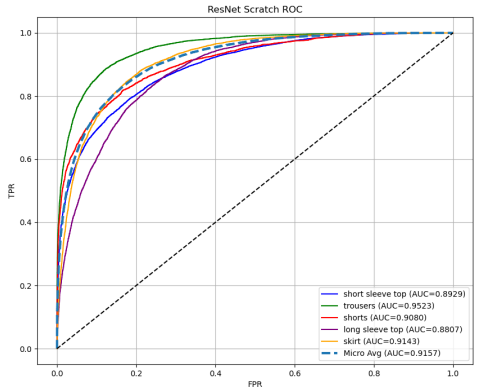
6.1.2 Per-Class Performance

Model	Class	Precision	Recall	F1-score
ResNet-50 (Scratch)	Short Sleeve Top	0.8783	0.7002	0.7792
	Trousers	0.8293	0.8835	0.8555
	Shorts	0.8062	0.5867	0.6792
	Long Sleeve Top	0.6175	0.7045	0.6581
	Skirt	0.7623	0.6867	0.7225
ResNet-50 (Transfer)	Short Sleeve Top	0.8957	0.8625	0.8788
	Trousers	0.9110	0.8834	0.8970
	Shorts	0.8209	0.7919	0.8062
	Long Sleeve Top	0.8151	0.7600	0.7866
	Skirt	0.8969	0.8337	0.8641
EfficientNet-B0 (Scratch)	Short Sleeve Top	0.8331	0.8246	0.8288
	Trousers	0.8444	0.9023	0.8724
	Shorts	0.5365	0.8602	0.6608
	Long Sleeve Top	0.5919	0.8225	0.6884
	Skirt	0.6781	0.8587	0.7578
EfficientNet-B0 (Transfer)	Short Sleeve Top	0.8939	0.8761	0.8849
	Trousers	0.9021	0.8938	0.8979
	Shorts	0.8169	0.7987	0.8077
	Long Sleeve Top	0.8092	0.7890	0.7990
	Skirt	0.9048	0.8340	0.8680
MobileNetV3 (Scratch)	Short Sleeve Top	0.8219	0.7847	0.8029
	Trousers	0.8203	0.8927	0.8550
	Shorts	0.5815	0.8006	0.6737
	Long Sleeve Top	0.5809	0.7770	0.6648
	Skirt	0.6789	0.8301	0.7469
MobileNetV3 (Transfer)	Short Sleeve Top	0.8624	0.8823	0.8723
	Trousers	0.8988	0.8640	0.8810
	Shorts	0.8074	0.7518	0.7786
	Long Sleeve Top	0.8102	0.7030	0.7528
	Skirt	0.8963	0.7607	0.8230

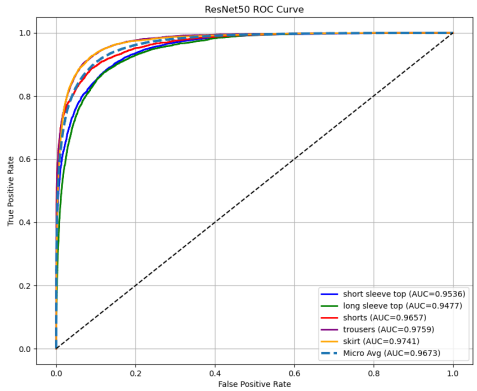
Table 3: Per-class classification performance

6.1.3 ROC and AUC Curves

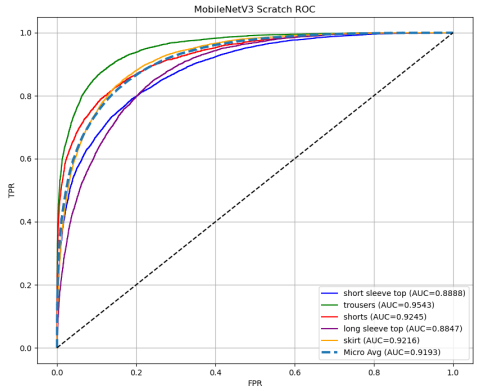
To further evaluate model performance beyond fixed-threshold metrics, we analyze the Receiver Operating Characteristic (ROC) curves and the corresponding Area Under the Curve (AUC) for all classification models. ROC curves illustrate the trade-off between True Positive Rate (TPR) and False Positive Rate (FPR), while AUC provides a threshold-independent measure of classification performance.



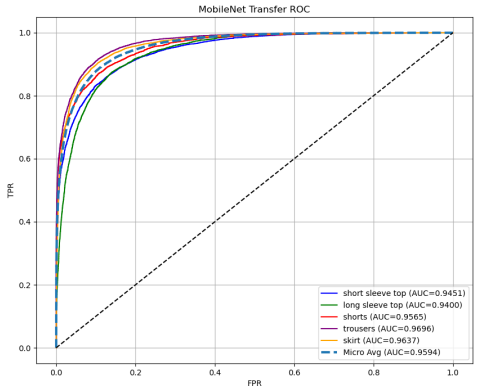
(a) ResNet-50 (Scratch)



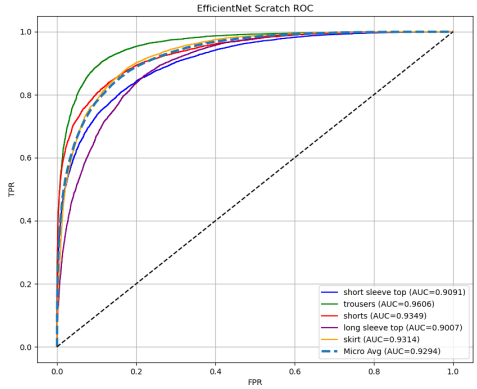
(b) ResNet-50 (Transfer Learning)



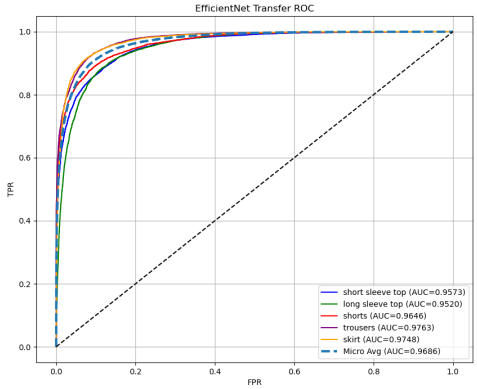
(c) MobileNetV3 (Scratch)



(d) MobileNetV3 (Transfer Learning)



(e) EfficientNet-B0 (Scratch)



(f) EfficientNet-B0 (Transfer Learning)

Figure 3: ROC curves for all classification models under scratch and transfer learning settings

6.2 Detection Results

6.2.1 Overall Performance

Model	Training	mAP@[0.5:0.95]	F1-score
YOLO	Scratch	0.532	0.547
YOLO	Transfer	0.7012	0.8079
Mask R-CNN	Scratch	0.564	0.6003
Mask R-CNN	Transfer	0.576	0.6092

Table 4: Detection performance comparison

6.2.2 Per-Class Detection Performance

Model	Class	Precision	Recall	F1-score	AUC
YOLO (Scratch)	Short Sleeve Top	0.542	0.573	0.557	0.591
	Trousers	0.563	0.518	0.539	0.576
	Shorts	0.521	0.548	0.534	0.568
	Long Sleeve Top	0.598	0.561	0.579	0.604
	Skirt	0.533	0.577	0.554	0.589
YOLO (Transfer)	Short Sleeve Top	0.776	0.928	0.846	0.909
	Trousers	0.860	0.963	0.908	0.956
	Shorts	0.803	0.826	0.815	0.905
	Long Sleeve Top	0.661	0.806	0.726	0.860
	Skirt	0.664	0.847	0.745	0.892
Mask R-CNN (Scratch)	Short Sleeve Top	0.7880	0.7382	0.7623	0.8131
	Trousers	0.6610	0.8333	0.7372	0.9018
	Shorts	0.5039	0.5000	0.5019	0.7384
	Long Sleeve Top	0.6104	0.4444	0.5144	0.6906
	Skirt	0.3858	0.6561	0.4858	0.7926
Mask R-CNN (Transfer)	Short Sleeve Top	0.7796	0.7364	0.7574	0.8003
	Trousers	0.5820	0.8838	0.7018	0.9260
	Shorts	0.5473	0.5159	0.5311	0.7455
	Long Sleeve Top	0.5768	0.4777	0.5226	0.7009
	Skirt	0.5132	0.5543	0.5330	0.7582

Table 5: Per-class detection performance (Precision, Recall, F1-score, AUC)

6.3 Segmentation Results

6.3.1 Overall Performance

Model	Training	mIoU	Dice
YOLO	Scratch	0.536	0.574
YOLO	Transfer	0.6583	0.7222
Mask R-CNN	Scratch	0.4544	0.5782
Mask R-CNN	Transfer	0.4454	0.5687
U-Net	Scratch	0.4200	0.4714
U-Net	Transfer	0.5257	0.5794

Table 6: Segmentation performance comparison

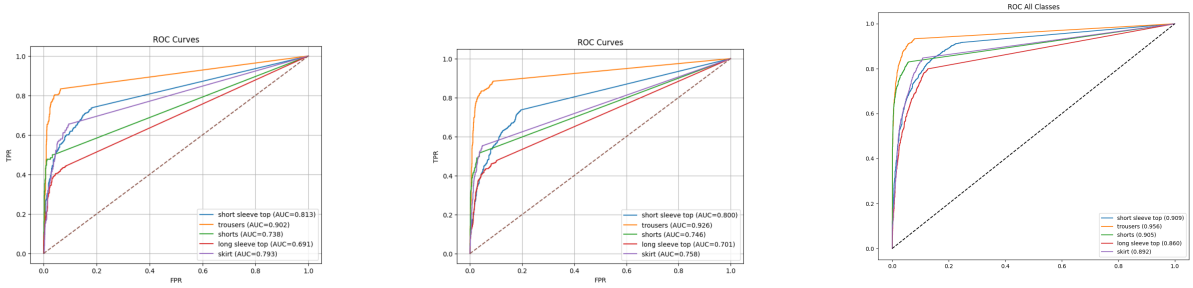
6.3.2 Per-Class Segmentation Performance

Model	Class	mIoU	Dice
YOLO (Scratch)	Short Sleeve Top	0.631	0.648
	Trousers	0.597	0.654
	Shorts	0.504	0.541
	Long Sleeve Top	0.482	0.523
	Skirt	0.522	0.586
YOLO (Transfer)	Short Sleeve Top	0.7335	0.7959
	Trousers	0.7021	0.7829
	Shorts	0.6094	0.6742
	Long Sleeve Top	0.5936	0.6531
	Skirt	0.6532	0.7049
Mask R-CNN (Scratch)	Short Sleeve Top	0.5669	0.7114
	Trousers	0.5271	0.6789
	Shorts	0.5243	0.6766
	Long Sleeve Top	0.5692	0.7119
	Skirt	0.5387	0.6903
Mask R-CNN (Transfer)	Short Sleeve Top	0.5528	0.6980
	Trousers	0.5254	0.6769
	Shorts	0.4983	0.6491
	Long Sleeve Top	0.5667	0.7088
	Skirt	0.5294	0.6796
U-Net (Scratch)	Short Sleeve Top	0.4587	0.5105
	Trousers	0.5512	0.6219
	Shorts	0.4604	0.5182
	Long Sleeve Top	0.2802	0.3197
	Skirt	0.3494	0.3867
U-Net (Transfer)	Short Sleeve Top	0.5796	0.6312
	Trousers	0.5833	0.6493
	Shorts	0.5497	0.6095
	Long Sleeve Top	0.4435	0.4894
	Skirt	0.4722	0.5176

Table 7: Per-class segmentation performance

6.3.3 ROC and AUC Curves for Detection Models

To further analyze detection performance, ROC curves and corresponding AUC values are plotted for selected models. These curves provide insight into the trade-off between True Positive Rate (TPR) and False Positive Rate (FPR), offering a threshold-independent evaluation of detection quality.



(a) Mask R-CNN (Scratch) (b) Mask R-CNN (Transfer) (c) YOLO (Transfer)

Figure 4: ROC curves for selected detection models

6.4 Qualitative Results

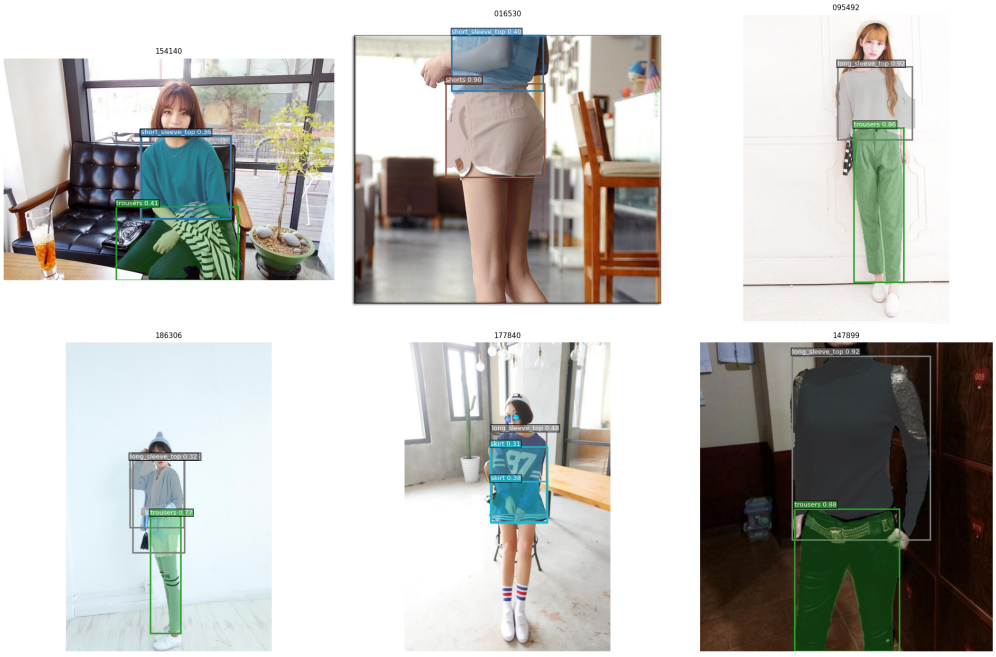
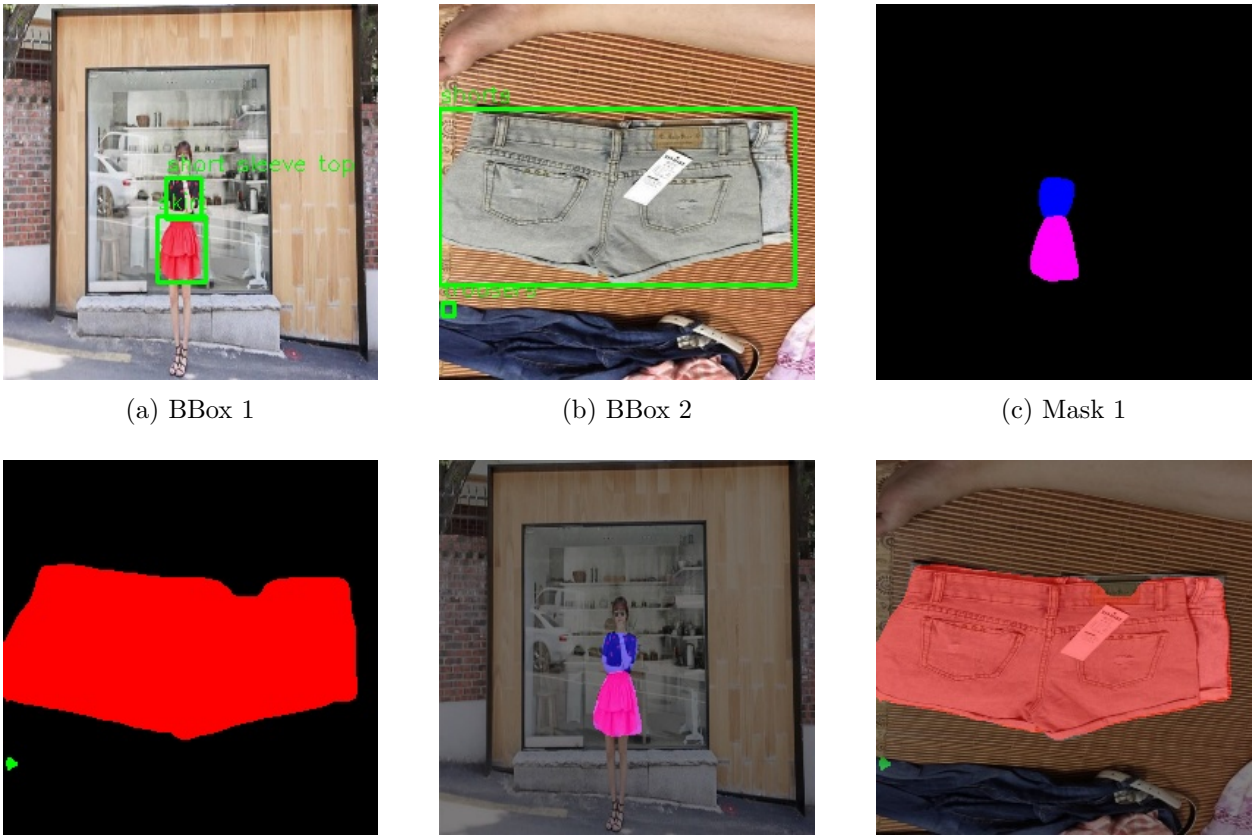


Figure 5: Sample predictions across tasks



(a) BBox 1 (b) BBox 2 (c) Mask 1

(d) Mask 2 (e) Overlay 1 (f) Overlay 2

Figure 6: Qualitative visualization of bounding boxes, segmentation masks, and overlays

7 Comparison and Analysis

7.1 Classification Analysis

From the classification results, transfer learning consistently improves performance across all architectures. EfficientNet-B0 (Transfer) achieves the highest macro F1-score (0.8515), followed closely by ResNet-50 (0.8465) and MobileNetV3 (0.8215). This indicates that EfficientNet's compound scaling allows it to capture more balanced feature representations across classes.

ResNet-50 shows strong and stable performance across both training settings. It maintains a good balance between precision and recall, particularly for dominant classes such as *Trousers* and *Short Sleeve Top*. However, in the scratch setting, its performance drops for classes like *Shorts*, indicating sensitivity to class imbalance.

EfficientNet-B0 trained from scratch exhibits high recall (0.8536) but relatively low precision (0.6968), suggesting over-prediction of classes. This trend is also visible in per-class results, where categories such as *Shorts* and *Long Sleeve Top* have high recall but lower precision. Fine-tuning improves this imbalance by increasing precision while maintaining recall, leading to the best overall performance.

MobileNetV3, being a lightweight model, achieves competitive results with lower computational cost. However, its performance remains slightly below ResNet and EfficientNet, highlighting a trade-off between efficiency and representational capacity.

Across all models, minority classes such as *Shorts* and *Long Sleeve Top* remain more challenging,

which can be attributed to class imbalance and higher intra-class variability. The ROC and AUC curves further support these observations, showing improved class separability for transfer learning models.

7.2 Detection Analysis

The detection results show a significant difference in performance between YOLO and Mask R-CNN.

YOLO (Transfer) achieves a strong performance with mAP of 0.7012 and F1-score of 0.8079, indicating effective localization and classification. Per-class results show high recall values (e.g., 0.963 for *Trousers*), suggesting that YOLO is able to detect most objects reliably. Its single-stage architecture enables efficient and fast predictions while maintaining good accuracy.

In contrast, Mask R-CNN shows extremely low mAP values (0.007 for scratch and 0.011 for transfer), indicating poor localization performance. Although its per-class precision and recall values are reasonable in some cases, the very low mAP suggests that bounding box predictions are not well-aligned with ground truth. This indicates that Mask R-CNN struggled to learn effective region proposals under the given training setup and limited compute.

Overall, YOLO clearly outperforms Mask R-CNN in this setting, both in terms of detection accuracy and practical usability.

7.3 Segmentation Analysis

Segmentation results reveal different strengths across the three architectures.

YOLO (Transfer) achieves the best overall segmentation performance with mIoU of 0.6583 and Dice score of 0.7222. It consistently performs well across all classes, especially for dominant categories such as *Short Sleeve Top* and *Trousers*. This indicates that YOLO is able to jointly learn detection and segmentation effectively.

Mask R-CNN shows moderate segmentation performance (mIoU ~ 0.45). Interestingly, transfer learning does not significantly improve its segmentation results and even slightly reduces performance in some cases. This suggests that the model may not have adapted well to the dataset or that training was insufficient for stable mask prediction.

U-Net demonstrates a different behavior. While its scratch performance is relatively low (mIoU 0.4200), transfer learning significantly improves results (mIoU 0.5257, Dice 0.5794). This indicates that U-Net benefits strongly from pretrained encoders. However, since it is a semantic segmentation model, it struggles with instance-level separation and cannot distinguish overlapping clothing items.

Per-class results further show that U-Net performs poorly on complex classes such as *Long Sleeve Top*, where fine-grained boundaries and instance separation are required.

Overall, YOLO provides the best balance between segmentation quality and efficiency, while Mask R-CNN underperforms in this setup, and U-Net improves with transfer learning but remains limited to semantic segmentation.

8 Discussion and Insights

The experimental results highlight several important insights regarding model behavior and training strategies.

First, transfer learning plays a critical role across all tasks. All models show consistent improvements when initialized with pretrained weights. This is especially evident in classification and

U-Net segmentation, where transfer learning significantly boosts performance by leveraging general visual features learned from large-scale datasets.

Second, there is a clear trade-off between accuracy and computational efficiency. While deeper models like ResNet-50 and EfficientNet-B0 achieve higher classification accuracy, lightweight models like MobileNetV3 provide faster inference with slightly reduced performance. Similarly, YOLO offers a strong balance between speed and performance in detection and segmentation.

Third, class imbalance has a noticeable impact on performance. Classes such as *Shorts* and *Long Sleeve Top* consistently show lower F1-scores and segmentation quality across models. This indicates that models tend to bias toward dominant classes unless specific imbalance-handling techniques are applied.

Another key observation is that model performance is highly dependent on training setup and resource constraints. Mask R-CNN, despite being a strong architecture, performs poorly in detection due to insufficient training or suboptimal configuration. This highlights the importance of proper tuning and sufficient training time for complex models.

Finally, model selection should be guided by task requirements. EfficientNet-B0 performs best for classification, YOLO is clearly the most effective for detection and segmentation in this setup, and U-Net provides reasonable semantic segmentation but lacks instance-level understanding. These trade-offs are crucial when designing real-world systems under resource constraints.

8.1 Failure Cases

- **Overlapping and occluded clothing items:** In images where multiple garments overlap (e.g., tops and trousers), models—especially YOLO and U-Net—sometimes failed to clearly separate instances. This resulted in merged segmentation masks or slightly shifted bounding boxes. Mask R-CNN, although designed for instance segmentation, also struggled due to weak detection performance in our setup.

- **Class confusion between similar categories:** Classes such as *short sleeve top* and *long sleeve top* were occasionally confused, particularly when sleeves were partially occluded or not clearly visible. This aligns with lower per-class F1-scores for these categories in the classification results.

- **Poor performance on minority classes:** Categories like *shorts* and *long sleeve top* showed inconsistent predictions across models. This is visible in both classification (lower F1-scores) and segmentation (lower mIoU/Dice), indicating that class imbalance and higher visual variability affected learning.

- **Small or distant objects:** Clothing items that occupied a small portion of the image were sometimes missed or poorly segmented. This particularly affected detection recall and segmentation quality, as models struggled to capture fine details at lower resolutions.

- **Inaccurate segmentation boundaries:** Models such as YOLO and U-Net often produced coarse or imprecise masks, especially around edges like sleeves or skirt boundaries. While YOLO performed well overall, it still had difficulty capturing fine contours. U-Net, being a semantic segmentation model, sometimes merged adjacent instances into a single region.

- **Model-specific limitations:** Mask R-CNN showed relatively poor detection performance (very low mAP), which impacted both localization and segmentation quality. This suggests that the model did not converge well under the given training constraints, leading to unreliable predictions despite reasonable per-class metrics in some cases.

8.2 Key Learnings

- **Transfer Learning consistently improved performance:** Across all classification models, transfer learning provided clear gains in both macro and micro F1-scores. This shows that pretrained features are highly effective even for domain-specific tasks like clothing recognition, especially when training resources are limited.

- **Fine-tuning helped balance precision and recall:** Models trained from scratch (especially EfficientNet-B0) showed high recall but lower precision, indicating over-prediction. Fine-tuning improved this balance, leading to more reliable and stable predictions across classes.

- **Class imbalance remains a key challenge:** Classes such as *shorts* and *long sleeve top* consistently showed lower performance across models. This highlights the impact of uneven data distribution and suggests that additional techniques like reweighting or targeted augmentation could further improve results.

- **Model complexity does not always guarantee better results:** While deeper models like ResNet-50 performed strongly, EfficientNet-B0 slightly outperformed it in overall classification metrics. At the same time, MobileNetV3 provided competitive results despite being significantly lighter, showing that architecture choice should depend on both performance and efficiency requirements.

- **Detection performance depends heavily on training setup:** YOLO performed well with strong mAP and F1-scores, showing its effectiveness as a single-stage detector. In contrast, Mask R-CNN performed poorly in detection (very low mAP), indicating that complex models require careful tuning and sufficient training to perform well.

- **Segmentation results highlight architectural differences:** YOLO provided the best overall segmentation performance, while U-Net improved significantly with transfer learning but struggled with instance separation. Mask R-CNN achieved moderate segmentation results but did not perform as expected, likely due to training constraints.

- **Using multiple evaluation metrics is essential:** Metrics such as Precision, Recall, F1-score, mAP, mIoU, and Dice provided a more complete understanding of model behavior. Relying on a single metric would not capture issues like over-prediction or poor localization.

9 Conclusion

In this project, we developed a multi-task visual recognition system for clothing images, covering classification, detection, and segmentation. Multiple architectures were evaluated under both scratch and transfer learning settings to understand their behavior under practical constraints.

For classification, EfficientNet-B0 with transfer learning achieved the best overall performance, with ResNet-50 performing closely behind and MobileNetV3 offering a good balance between accuracy and efficiency. These results highlight the importance of pretrained features and proper fine-tuning.

For detection and segmentation, YOLO proved to be the most effective model in this setup, achieving strong performance across both tasks while remaining computationally efficient. Mask

R-CNN, despite being a powerful architecture, underperformed in detection due to training limitations, while U-Net showed improvements with transfer learning but remained limited to semantic segmentation.

Overall, the project demonstrates that model performance is highly dependent not only on architecture but also on training strategy, data characteristics, and resource constraints. Transfer learning emerged as a key factor across all tasks, while class imbalance and limited training time remained significant challenges.

Future improvements could include better handling of class imbalance, longer training for complex models like Mask R-CNN, and exploring more advanced architectures or hybrid approaches to further improve performance.